AI - Lab

6. Natural Language Processing with RNNs



Hochschule Karlsruhe
Technik und Wirtschaft
UNIVERSITY OF APPLIED SCIENCES

Dr. Fabian Sigloch

WS 26/27

From images to language

Hochschule Karlsruhe University of Applied Sciences HKA

Weeks 3–5: images

image $\longrightarrow$ features $\longrightarrow$ class / boxes

- A CNN processes a grid of pixels.
- Convolutions reuse the same filters across locations.

This week: text

words $\longrightarrow$ sequence model $\longrightarrow$ label

- A sentence is a variable-length sequence.
- Word order and context can change meaning.

We will turn text into numbers, then learn a model that reads it in order.

Natural language processing

Hochschule Karlsruhe University of Applied Sciences HKA

NLP is the set of methods that let computers work with human language.

Common tasks

- Text or sentiment classification
- Named-entity recognition
- Part-of-speech tagging
- Translation and summarisation

Where we meet them

- Search and spam filtering
- Voice assistants and chatbots
- Social media analysis
- Writing and translation tools

This lab focuses on text classification: map one text sequence to one category.

Text classification: emotion in a tweet

Hochschule Karlsruhe University of Applied Sciences HKA

Input

"My roommate: it's okay that we can't spell because we have autocorrect.#terrible"



- The model predicts one label from a fixed set of classes.
- TweetEval is a benchmark containing several Twitter classification tasks, including emotion recognition.
- The label is a training target; the model must infer it from the words and their context.

A neural network needs numbers

Hochschule Karlsruhe University of Applied Sciences HKA

Text

"cats chase mice"

Tokens

$[\texttt{cats}, \texttt{chase}, \texttt{mice}]$

Vocabulary and token IDs

$\texttt{cats} \mapsto 17, \quad \texttt{chase} \mapsto 42, \quad \texttt{mice} \mapsto 81$

Sequence of IDs

$[17, 42, 81]$

Vocabulary

A vocabulary assigns each known token a unique integer. Unknown or rare words need a special `<unk>` token.

One-hot encoding: possible, but inefficient

Hochschule Karlsruhe University of Applied Sciences HKA

Suppose the vocabulary is $V = 6$ words. Each token becomes a vector of length six:

$$\texttt{cats} \mapsto [0, 0, 1, 0, 0, 0], \qquad \texttt{mice} \mapsto [0, 0, 0, 0, 1, 0].$$

- Only one entry is 1; all other entries are 0.
- Real vocabularies may contain tens of thousands of tokens, so vectors are large and sparse.
- The representation does not say that "cat" and "kitten" are related.

Key limitation

Different words are unrelated coordinate directions in one-hot space.

Embeddings: compact word vectors

Hochschule Karlsruhe University of Applied Sciences HKA

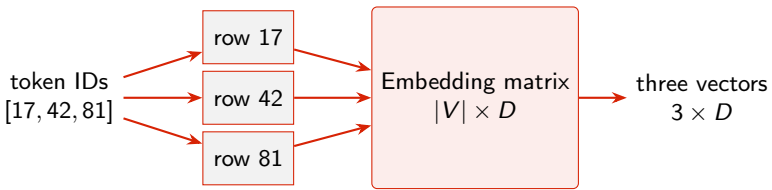
An embedding maps each token ID to a short, dense vector:

$$17 \mapsto [0.2, -0.7, 0.1, \ldots], \qquad 81 \mapsto [-0.1, 0.4, 0.8, \ldots]$$

- Instead of a vector as long as the vocabulary, each word gets D learned numbers.
- During training, the embedding table is updated to help the task.
- Useful representations can place words used in similar contexts near each other.
- Pre-trained embeddings such as Word2Vec or GloVe can be reused; this assignment learns embeddings from its training data.

Token IDs are lookup keys; the embedding vectors are the values supplied to the sequence model.

The embedding layer is a lookup table



- `nn.Embedding(num_embeddings=|V|, embedding_dim=D)` stores one trainable row per vocabulary item.
- Looking up a sentence returns one vector per token, preserving sequence order.

Why use a sequence model?

Hochschule Karlsruhe
University of
Applied Sciences
HKA

Order changes meaning

"dog bites person"

"person bites dog"

The same words can express different events when their order changes.

Length varies

"great"

"the film was not great at all"

Sentences do not all have the same number of tokens.

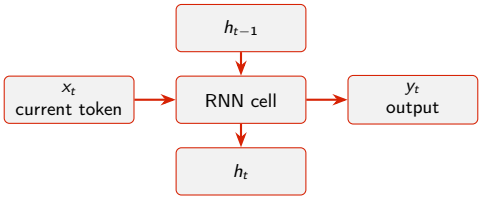
- A fixed-size fully connected input needs padding or truncation and does not naturally share computation over time.
- Recurrent networks read tokens in sequence and carry a state forward.

RNNs read tokens one at a time

Hochschule Karlsruhe University of Applied Sciences HKA

At time step t, the RNN combines the current word vector x_t with its previous state h_{t-1}:

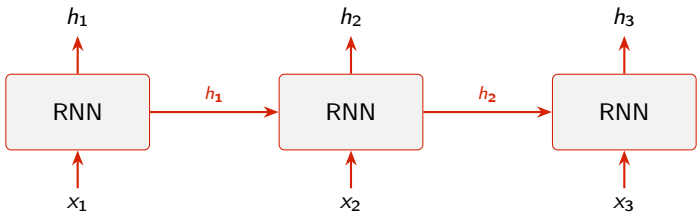
$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h), \qquad y_t = W_{hy}h_t + b_y.$$



Example sizes for this assignment

With embedding dimension $D = 128$, hidden size $H = 256$, and $C = 4$ classes: $x_t \in \mathbb{R}^{128}$, $h_{t-1}, h_t, b_h \in \mathbb{R}^{256}$; $W_{xh} \in \mathbb{R}^{256\times128}$, $W_{hh} \in \mathbb{R}^{256\times256}$; $W_{hy} \in \mathbb{R}^{4\times256}$, $b_y \in \mathbb{R}^4$. Both terms before tanh have 256 components, so they can be added.

Unrolling the RNN over a sentence



- Each step processes the next token and passes its state to the next step.
- The same matrices are reused at every time step: one model handles short and long sequences.
- For sentence classification, the final state can be used to predict one label for the whole sentence.

Long sequences challenge basic RNNs

Hochschule Karlsruhe University of Applied Sciences HKA

- Training unfolds the recurrence across all tokens; gradients flow backward through these steps.
- Repeated multiplication by derivatives can make a gradient shrink or grow rapidly.
- When it shrinks, early words receive little learning signal: the **vanishing-gradient problem**.
- This makes it difficult for a basic RNN to learn relationships between far-apart words.

Example

"The book, which I borrowed from the library last week, was excellent."
The subject and verb are separated by several words.

LSTM and GRU cells add gates that help the network decide what information to keep or update.

LSTM: a gated memory path

Hochschule Karlsruhe University of Applied Sciences HKA

An LSTM maintains a cell state c_t in addition to its hidden state h_t.

$$
\begin{aligned}
f_t &= \sigma(W_f[x_t, h_{t-1}] + b_f) && \text{forget gate} \\
i_t &= \sigma(W_i[x_t, h_{t-1}] + b_i) && \text{input gate} \\
\tilde{c}_t &= \tanh(W_c[x_t, h_{t-1}] + b_c) && \text{candidate memory} \\
o_t &= \sigma(W_o[x_t, h_{t-1}] + b_o) && \text{output gate}
\end{aligned}
$$

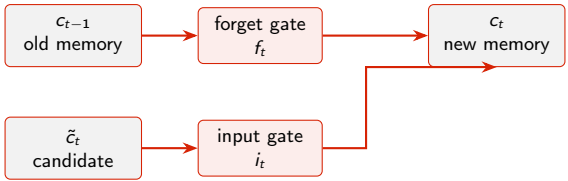
$$
c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \qquad h_t = o_t \odot \tanh(c_t)
$$

Intuition

- Forget: what old memory can be reduced?
- Input: what new information should be stored?
- Output: what part of memory should be exposed now?

σ outputs values between 0 and 1; $\odot$ is element-wise multiplication. The gates are learned from data.

The LSTM memory path



- A gate is a learned, element-wise control signal; it is not a hand-written rule.
- This memory path helps an LSTM retain useful information over longer sequences than a basic RNN.
- LSTM does not guarantee that every long-range relationship will be learned.

GRU: a simpler gated recurrent unit

Hochschule Karlsruhe University of Applied Sciences HKA

GRU gates

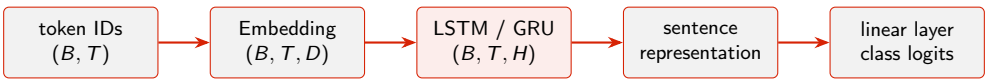
- **Update gate**: how much of the old state to retain versus replace.
- **Reset gate**: how much earlier context to use when forming new content.
- The hidden state is also the memory; there is no separate LSTM cell state.

Compared with an LSTM

- Fewer gates and parameters.
- Often faster to train.
- May perform similarly; which is better depends on data and setup.

Both LSTM and GRU are recurrent models with learned gates; neither is universally best.

Build a text classifier



1. Convert words to token IDs, then look up embeddings.
2. Read the sequence with an LSTM or GRU.
3. Use the final valid state to classify the whole sequence.

B: batch size; T: padded sequence length; D: embedding dimension; H: hidden size.

Tensor shapes in PyTorch

Hochschule Karlsruhe University of Applied Sciences HKA

With `batch_first=True`, an embedding sequence has shape:

$$\underbrace{(B, T, D)}_{\text{batch, time steps, embedding features}} \xrightarrow{\texttt{nn.LSTM}} \underbrace{\text{output } (B, T, H)}_{\text{state at every time step}}, \quad \underbrace{h_n : (L, B, H)}_{\text{final state per layer}}.$$

- B is the number of examples processed together.
- T is the number of tokens after padding or truncation.
- L is the number of recurrent layers; a bidirectional model also changes the final dimension.
- For a one-layer, unidirectional classifier, use `h_n[-1]` as the sentence representation.

Padding matters

The final time step of a padded batch may be padding, not a real word. Use lengths/packed sequences or select each example's last valid state.

Training a multi-class text classifier

Hochschule Karlsruhe University of Applied Sciences HKA

Model output

- One logit per class.
- Logits are unnormalised scores.
- The largest logit gives the predicted class.

Loss and update

- Compare logits with the integer class label using `CrossEntropyLoss`.
- Backpropagate through the classifier, recurrent layer, and embedding table.
- Update parameters with an optimizer such as Adam.

Do not apply softmax before PyTorch cross entropy

`CrossEntropyLoss` combines log-softmax and negative log-likelihood internally.

Evaluation: accuracy is only a starting point

Hochschule Karlsruhe University of Applied Sciences HKA

- Test accuracy is the fraction of tweets assigned the correct label.
- A confusion matrix shows which emotions the model confuses.
- Report the same held-out test split for LSTM and GRU; use validation data for model choices.
- Compare training curves as well as the final score: a model can fit training data while generalising poorly.

Keep the comparison fair

Use the same data split, vocabulary, preprocessing, embedding dimension, hidden dimension, and evaluation procedure for both models.

Emotion labels can be subjective; model accuracy reflects agreement with dataset labels, not a definitive reading of a person's feelings.

Assignment 6: classify TweetEval emotions

Hochschule Karlsruhe University of Applied Sciences HKA

Goal: compare LSTM and GRU models on the TweetEval emotion-recognition task.

1. Tokenize the tweets and build a vocabulary from training data only; cap it at the 3,000 most frequent words and map other words to `<unk>`.
2. Implement the same embedding–recurrent–linear classifier twice: once with an LSTM and once with a GRU.
3. Use embedding dimension 128 and hidden dimension 256. For sequence classification, compute the loss from the final valid sequence representation.
4. Train and evaluate both models with the same split. Report test accuracy and discuss the comparison.

Preprocessing choice

Follow the supplied tutorial's tokenization instructions. If removing stop words, preserve negations such as "not"; otherwise sentiment can be reversed.

Resources: TweetEval dataset | PyTorch sequence models | spaCy introduction

Batching variable-length text

Hochschule Karlsruhe University of Applied Sciences HKA

Core task

- A simple implementation may process one tweet at a time.
- This is easy to understand but leaves much of the GPU idle.

Optional extension

- Group examples in a `DataLoader`.
- Pad each batch to its longest tweet and keep the true lengths.
- Train on GPU; compare runtime with single-example training.

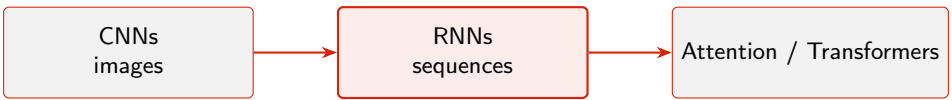
Why a custom `collate_fn`?

Text sequences have different lengths, so the loader must pad them before they can be stacked into one tensor.

Mask or pack padded sequences so padding tokens do not become part of the sentence representation.

From recurrence to attention

- RNNs build a representation one token at a time and pass a state along the sequence.
- LSTM and GRU gates help preserve useful context, but computation remains sequential across tokens.
- Attention lets a model relate tokens directly to other tokens in the sequence.
- Transformer models build on attention and can create contextual representations for every token.



Takeaways

Hochschule Karlsruhe University of Applied Sciences HKA

- Text becomes a sequence of token IDs; embeddings turn IDs into learned dense vectors.
- RNNs reuse the same cell across time and carry a hidden state through the sequence.
- LSTM and GRU gates help manage information over longer sequences.
- For text classification, a sequence representation feeds a classifier trained with cross-entropy.
- Fair evaluation requires consistent preprocessing, splits, and metrics.

Happy coding!